

-



- Processing Data

On this page

Processing Data

Was this page helpful? 

Now that you have loaded your data into Pulse, the next step is to transform your data and extract value from it. At this stage you will learn how to design [plans](#) so that you can design derived features or extract subsets of your original data set, before training a model.

Designing derived features

It is often useful to enrich your dataset with derived features that can be useful to distinguish between the classes you wish your model to predict (in this case, *fraud* and *not fraud*). Design a plan that calculates a feature as follows:

1. In the **Data Science** tab, add a new plan named *Features*.

2. Set the plan as **Deployable**. Generating derived features is useful both in data science and in production [workflows](#). Enabling this option allows you to take this work directly into the runtime environment without any extra effort.
3. In the **Schema** field, set the schema to be copied from the *RawTransactions* data source (confirm the orange warning sign below) and **Save** it.
4. Use a **Map** operator to calculate derived features. You can design a feature that represents the distance between the merchant and the customer's home location (since a greater distance has a higher correlation with fraud). Create a **Map** named *AddDistance* to add a new *distance* field to your data and configure it as follows:

Designing historical profiles

Another kind of features that is also often useful are the ones based on [historical behaviour](#). To calculate historical profiles (i.e. metrics), such as the daily count and sum of transactions, execute the following actions:

1. In the *Features* plan, use the button to select other options, and select the **Plan Settings** option.

Under **Timestamp Field**, select the field called *Timestamp*. This is required so that Pulse knows which field from the dataset represents when events occurred so that it can build time-based features.

2. Add a **Metrics** operator and connect it after the *AddDistance* operator.
3. Edit the metrics operator to name it *Metrics* and check the **Output Incomplete Instances** option.

Initial events will not have accurate metrics. For example, a metric with a window of 24 hours will only be correct after the first 24 hours of the data. Hence these events are called incomplete instances. By default, Pulse discards the initial events with incomplete metrics, to improve model training and evaluation. Select the **Output Incomplete Instances** option if you want to include them anyways.

4. Add a new metric, and define it as follows:

1. In the **Group By** section, select the *AccountID* field. This means that the metric will be calculated for each account.
2. In the **Window** section, leave the default *1 hour*. This means that you want to calculate a metric based on the events of the last hour.
3. In the **Aggregation** section, select the function *Count*. This means that the metric calculates the number of events in the respective window (last hour).
4. Use the **Save** button to add the metric to the operator.

5. Add another metric following the same steps, but this time the window is *1 day*. This metric calculates the number of events in the last day.

6. Add a new metric, and define it as follows:

1. In the **Group By** section, select the *AccountID* field.
2. In the **Window** section, leave the default *1 hour*.
3. In the **Aggregation** section, select the function *Sum*, and the field *TransactionAmount*. This means that the metric calculates the total amount of the events in the respective window (last hour).
4. Use the **Save** button to save the metrics.

7. Add another metric following the same steps, but this time the window is *1 day*. This metric calculates the total amount of the events in the last day.

8. Use the button next to the metrics' names to change them to *HourlyTrCount*, *DailyTrCount*, *HourlyTrSum*, and *DailyTrSum*, respectively.

9. Use the **Save** button to save the metrics operator. Your feature plan will look like the following image:

10. Use the button to run the plan and create a new and enhanced data source including your features. Configure the execution as follows:

1. In the **Data Source** section, select *RawTransactions* as the input data source.

2. In the **Data Sinks** tab, name the data sink as `TransactionsFeatures`, and leave the file type as the default (Parquet). In the **Directory Path** field, specify the output file URL. Set the path to the Pulse server home directory, ending with `/var/datasources/<filename>`.

Example: `file:///PULSE_SERVER_HOME/var/datasources/transactions-features.parquet`

The file path may vary according to the operating system of the Pulse server. You can not have a file with the same name in the `datasources` folder.

3. Use the **Run** button to execute the plan.

11. **Save** the plan and then press the **refresh** button on browser.

You can see in the **Data Science** tab that you now have a new data source. You can analyze it as you have done before with *RawTransactions*, and assess the usefulness of the features you have designed.

Sampling your data

It is often useful, in the context of Data Science, to reduce the size of the dataset while generally maintaining the distributions of the random variables representing each feature. This is done via [stratified sampling](#).

Sampling improves the performance of your Data Science Loop since you will be able to iterate much faster on feature calculation and model training.

Create a stratified sample to reduce the number of *Accounts* where there was no *Fraud* to be 10% of its original size, as follows:

1. Add a new plan named *Stratify*.

Make sure that the **Deployable** checkbox is **not** selected. This means that this plan will be non-deployable. Non-deployable plans are useful for sampling data, which typically happens only in the data science environment, hence the plan won't need to be deployed to production.

2. Add a new **Source** operator named *Transactions* that copies the *RawTransactions* schema.
3. Calculate the number of *Fraud* transactions per *Account*. First, add a **Map** operator, and connect it to the `source**, **` with the following configuration:
4. Sum up the number of *Fraud Transactions* by adding a **Group By** operator like the one below. This way, you will have the total of *Fraud* transactions per account stored in a field named *nrFrauds*.
5. Split the data set into two: accounts where there was fraud and accounts where there wasn't. Add a **Filter** operator as follows:
6. Repeat the previous step, but this time with a *Fraudulent* filter (use *nrFrauds > 0* as the expression).
7. Apply a **Sample** operator to the *Legitimate* transactions to get only 10% of them.
8. Join both sets from *Fraudulent* and the reduced *Legitimate* with a **Union** operator.
9. Use a **Join** operator to join the result with the original *Transactions*, so that you end up with the original data set minus 90% of the *Transactions* belonging to an *Account* where there was no *Fraud*.
10. Finally, add a **Sink** operator.

You can run this plan as you have done with the *Features* plan to generate a new data source (do not forget to select the *RawTransactions* as your base data source). You can then verify, by evaluating it and performing an exploratory data analysis, if this sample really is representative of the original data set. For most data sets, this is true. You will not be using this sample (or any other) during this tutorial since the original data set is fairly small already. This allows you to operate on it without resorting to sampling.

However, it is encouraged that you to use this technique on real projects with substantial data sets.